

1. WHAT IS FREERTOS?

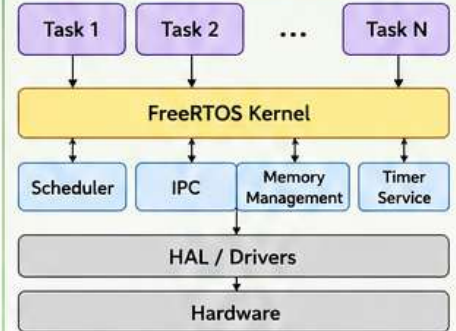
- FreeRTOS is a free, open source, real-time operating system (RTOS) for microcontrollers.
- Written in C, portable, small, and efficient.
- Supports multitasking, task scheduling, synchronization, communication and timing management.



2. KEY FEATURES

- ✓ Preemptive multitasking
- ✓ Configurable task priority
- ✓ Low memory footprint
- ✓ Deterministic real-time behavior
- ✓ Portability across architectures
- ✓ Rich set of APIs
- ✓ Tickless idle & low power support
- ✓ Extensive documentation & community

3. FREERTOS ARCHITECTURE

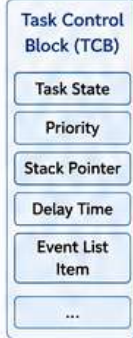


4. TASKS (THREADS)

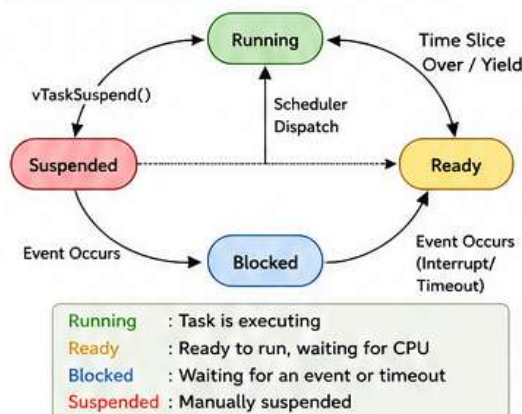
- A task is a basic unit of execution.
- Each task has its own stack.
- Tasks communicate using queues, semaphores, etc.
- Created dynamically or statically.

```

TaskHandle_t xTaskCreate(
    TaskFunction_t pxTaskCode,
    const char * const pcName,
    const uint32_t ulStackDepth,
    void *pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pxCreatedTask
);
  
```



5. TASK STATES



6. TASK PRIORITY

- Higher priority task runs before lower priority.
- Preemptive scheduling.
- Configurable priority levels (configMAX_PRIORITIES).

Priority Example (Higher number = Higher priority)

Priority 3	→ Task A (Running)
Priority 2	→ Task B (Ready)
Priority 1	→ Task C (Ready)
Priority 0	→ Task D (Blocked)

7. INTER TASK COMMUNICATION (IPC)



8. TIME MANAGEMENT



- System Tick (SysTick) – Heartbeat of the kernel.
- configTICK_RATE_HZ defines tick frequency.
- Delay APIs:
 - vTaskDelay(ticks)
 - vTaskDelayUntil(&lastWakeTime, ticks)
 - xTaskGetTickCount()
 - xTaskGetTickCountFromISR()

9. SCHEDULER

- Preemptive scheduler.
- Chooses the highest priority ready task.
- Time slicing for tasks with same priority (if enabled).
- Triggered by:
 - SysTick interrupt
 - Task yield
 - Blocking API
 - Higher priority task unblocked



10. MEMORY MANAGEMENT

- Dynamic: pvPortMalloc(), vPortFree()
- Static: xTaskCreateStatic(), xQueueCreateStatic(), etc.
- Heap schemes:
 - heap_1 (simple)
 - heap_2 (best fit)
 - heap_3 (malloc wrapper)
 - heap_4 (best fit with coalescing)
 - heap_5 (multiple regions)



11. INTERRUPT SUPPORT

- ISRs can be used with FreeRTOS API (FromISR variants).
- Use xQueueSendFromISR(), xSemaphoreGiveFromISR(), etc.
- Use portYIELD_FROM_ISR() to request context switch.



12. SOFTWARE TIMERS

- Built on Timer Service Task.
- Types:
 - One-shot Timer
 - Auto-reload Timer
- APIs:
 - xTimerCreate()
 - xTimerStart()
 - xTimerStop()
 - xTimerDelete()



13. HOOK FUNCTIONS

- vApplicationIdleHook()
- vApplicationTickHook()
- vApplicationMallocFailedHook()
- vApplicationStackOverflowHook()
- vApplicationDaemonTaskStartupHook()



14. MEMORURATION (FreeRTOSConfig.h)

- configTICK_RATE_HZ
- configMAX_PRIORITIES
- configMINIMAL_STACK_SIZE
- configUSE_PREEMPTION
- configUSE_MUTEXES
- configUSE_TIMERS
- and more...



15. DEBUG & TRACE

- Use configASSERT() for runtime checks.
- Trace tools:
 - Percepio Tracealyzer
 - SEGGER SystemView
- Helps in performance analysis and debugging.



16. BEST PRACTICES

- ✓ Keep ISRs short
- ✓ Avoid blocking in high priority tasks
- ✓ Use appropriate stack size
- ✓ Use queues/semaphores for communication
- ✓ Test thoroughly



IMPORTANT API CATEGORIES

Task Management	Task Control	Synchronization	Communication	Timers	System
xTaskCreate() vTaskDelete() vTaskSuspend() vTaskResume()	vTaskDelay() vTaskDelayUntil() taskYIELD() uxTaskPriorityGet()	xSemaphoreCreateBinary() xSemaphoreGive() xSemaphoreTake() xSemaphoreGiveFromISR()	xQueueCreate() xQueueSend() xQueueReceive() xQueueSendFromISR()	xTimerCreate() xTimerStart() xTimerStop() xTimerDelete()	xTaskGetTickCount() pvPortMalloc() vPortFree() configASSERT()

FreeRTOS – Important APIs Cheat Sheet

Most Commonly Used APIs in FreeRTOS



1. KERNEL CONTROL

- vTaskStartScheduler()
- vTaskEndScheduler()
- xTaskGetSchedulerState()
- vTaskSuspendAll()
- xTaskResumeAll()
- xTaskGetTickCount()
- xTaskGetTickCountFromISR()
- vTaskDelay()
- vTaskDelayUntil()
- taskYIELD()



2. TASK MANAGEMENT

Task Creation

- xTaskCreate()
- xTaskCreateStatic()

Task Delete

- vTaskDelete()

Suspend / Resume

- vTaskSuspend()
- vTaskResume()
- xTaskResumeFromISR()

Priority

- vTaskPrioritySet()
- uxTaskPriorityGet()

Information

- xTaskGetCurrentTaskHandle()
- pcTaskGetName()
- eTaskGetState()
- uxTaskGetStackHighWaterMark()



3. QUEUE APIs

Create

- xQueueCreate()
- xQueueCreateStatic()

Send

- xQueueSend()
- xQueueSendToBack()
- xQueueSendToFront()
- xQueueOverwrite()
- xQueueSendFromISR()
- xQueueSendToBackFromISR()
- xQueueSendToFrontFromISR()
- xQueueOverwriteFromISR()

Receive

- xQueueReceive()
- xQueuePeek()
- xQueueReceiveFromISR()
- xQueuePeekFromISR()

Utilities

- uxQueueMessagesWaiting()
- uxQueueSpacesAvailable()
- xQueueReset()
- vQueueDelete()



4. SEMAPHORE APIs

Create

- xSemaphoreCreateBinary()
- xSemaphoreCreateCounting()
- xSemaphoreCreateMutex()
- xSemaphoreCreateRecursiveMutex()

Take / Give

- xSemaphoreTake()
- xSemaphoreGive()
- xSemaphoreTakeRecursive()
- xSemaphoreGiveRecursive()

From ISR

- xSemaphoreGiveFromISR()
- xSemaphoreTakeFromISR()



5. EVENT GROUP APIs

- xEventGroupCreate()
- xEventGroupWaitBits()
- xEventGroupSetBits()
- xEventGroupSetBitsFromISR()
- xEventGroupClearBits()
- xEventGroupClearBitsFromISR()
- xEventGroupSync()
- xEventGroupGetBits()
- vEventGroupDelete()



6. SOFTWARE TIMER APIs

Create

- xTimerCreate()
- xTimerCreateStatic()

Control

- xTimerStart() / xTimerStartFromISR()
- xTimerStop() / xTimerStopFromISR()
- xTimerReset() / xTimerResetFromISR()
- xTimerChangePeriod()
- xTimerChangePeriodFromISR()

Other

- xTimerDelete()
- xTimerIsTimerActive()
- pcTimerGetName()



7. TASK NOTIFICATION APIs

- xTaskNotify()
- xTaskNotifyWait()
- xTaskNotifyGive()
- ulTaskNotifyTake()
- xTaskNotifyStateClear()
- ulTaskNotifyValueClear()
- xTaskNotifyFromISR()
- vTaskNotifyGiveFromISR()

Indexed Notifications

- xTaskNotifyIndexed()
- xTaskNotifyWaitIndexed()
- ulTaskNotifyTakeIndexed()
- xTaskNotifyIndexedFromISR()



8. STREAM & MESSAGE BUFFER

Stream Buffer

- xStreamBufferCreate()
- xStreamBufferSend()
- xStreamBufferReceive()
- xStreamBufferSendFromISR()
- xStreamBufferReceiveFromISR()
- xStreamBufferReset()
- xStreamBufferBytesAvailable()

Message Buffer

- xMessageBufferCreate()
- xMessageBufferSend()
- xMessageBufferReceive()
- xMessageBufferSendFromISR()
- xMessageBufferReceiveFromISR()
- xMessageBufferReset()
- xMessageBufferSpaceAvailable()



9. MEMORY MANAGEMENT

- pvPortMalloc()
- vPortFree()
- xPortGetFreeHeapSize()
- xPortGetMinimumEverFreeHeapSize()



10. CRITICAL SECTION

- taskENTER_CRITICAL()
- taskEXIT_CRITICAL()
- taskENTER_CRITICAL_FROM_ISR()
- taskEXIT_CRITICAL_FROM_ISR()



11. QUEUE SET APIs

- xQueueCreateSet()
- xQueueAddToSet()
- xQueueRemoveFromSet()
- xQueueSelectFromSet()
- xQueueSelectFromSetFromISR()



12. MISCELLANEOUS

Co-routine (Legacy)

- xCoRoutineCreate()
- crSTART() • crEND()
- crDELAY()
- crQUEUE_SEND()
- crQUEUE_RECEIVE()

Thread Local Storage

- vTaskSetThreadLocalStoragePointer()
- pvTaskGetThreadLocalStoragePointer()



13. ESP-IDF / DUAL CORE SPECIFIC APIs (ESP32 / ESP32-S3)

- xTaskCreatePinnedToCore()
- xTaskCreateStaticPinnedToCore()
- xTaskGetCoreID()
- vTaskCoreAffinitySet()
- vTaskCoreAffinityGet()



MOST FREQUENTLY USED (TOP 15)

- | | | |
|-----------------------|----------------------------|---------------------------|
| 1. xTaskCreate() | 6. xQueueCreate() | 12. xEventGroupSetBits() |
| 2. vTaskDelete() | 7. xQueueSend() | 13. xEventGroupWaitBits() |
| 3. vTaskDelay() | 8. xQueueReceive() | 14. xTimerCreate() |
| 4. vTaskDelayUntil() | 9. xSemaphoreCreateMutex() | 15. xTaskNotifyGive() |
| 5. vTaskPrioritySet() | 10. xSemaphoreTake() | ulTaskNotifyTake() |



NOTES

- ISR safe APIs are marked with "FromISR".
- Some APIs may require enabling options in FreeRTOSConfig.h
- For full details, refer to FreeRTOS API Reference Manual.

FreeRTOS SEMAPHORE – Explained with Examples

What is Semaphore?

A semaphore is a **synchronization** primitive used to control access to a **shared resource** and to **signal** between tasks or between an ISR and a task.

How it works?

- It has a count value.
- Tasks can **TAKE** (decrement) the semaphore. If count is 0, the task blocks (waits).
- Tasks/ISR can **GIVE** (increment) the semaphore. If tasks are waiting, one is unblocked.

API Summary

<code>xSemaphoreCreateBinary()</code>	– Create binary semaphore
<code>xSemaphoreCreateCounting(max, init)</code>	– Create counting semaphore
<code>xSemaphoreCreateMutex()</code>	– Create mutex (binary semaphore with priority inheritance)
<code>xSemaphoreTake(sem, timeout)</code>	– Take/Give (wait)
<code>xSemaphoreGive(sem)</code>	– Give (signal)
<code>xSemaphoreGiveFromISR(sem, &hwp)</code>	– Give from ISR
<code>xSemaphoreTakeFromISR(sem, &hwp)</code>	– Take from ISR

Types of Semaphores

1. Binary Semaphore

- Value is 0 or 1.
- Used for event signaling or simple resource control.



2. Counting Semaphore

- Value ranges from 0 to max count.
- Used to manage multiple identical resources.



3. Mutex (Binary Semaphore with Priority Inheritance)

- Used for mutual exclusion (one task at a time).
- Has priority inheritance to avoid priority inversion.



EXAMPLES WITH CODE & OUTPUT

Example 1: Binary Semaphore – Task Signaling

Scenario: Task B waits for a signal from Task A.

```
SemaphoreHandle_t binSem;

void TaskB(void *pv){
    printf("[B] Waiting for signal...\n");
    xSemaphoreTake(binSem, portMAX_DELAY); // Wait
    printf("[B] Received signal!\n");
    vTaskDelete(NULL);
}

void TaskA(void *pv){
    vTaskDelay(pdMS_TO_TICKS(2000));
    printf("[A] Sending signal...\n");
    xSemaphoreGive(binSem); // Signal to B
    vTaskDelete(NULL);
}

void app_main(void){
    binSem = xSemaphoreCreateBinary();
    xTaskCreate(TaskB, "TaskB", 2048, NULL, 1, NULL);
    xTaskCreate(TaskA, "TaskA", 2048, NULL, 1, NULL);
}
```

Output (Console)

```
[B] Waiting for signal...
[A] Sending signal...
[B] Received signal!
```

Example 2: Counting Semaphore – Resource Pool

Scenario: 3 identical resources available.

```
SemaphoreHandle_t countSem;

void Worker(void *pv){
    int id = (int)pv;
    printf("[Worker %d] Waiting for resource...\n", id);
    xSemaphoreTake(countSem, portMAX_DELAY);
    printf("[Worker %d] Got resource!\n", id);
    vTaskDelay(pdMS_TO_TICKS(2000)); // Using resource
    printf("[Worker %d] Released resource!\n", id);
    xSemaphoreGive(countSem);
    vTaskDelete(NULL);
}

void app_main(void){
    countSem = xSemaphoreCreateCounting(3, 3); // max=3, init=3
    for(int i=1; i<=5; i++){
        xTaskCreate(Worker, "Worker", 2048, (void*)i, 1, NULL);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

Output (Console) (order may vary)

```
[Worker 1] Waiting for resource...
[Worker 1] Got resource!
[Worker 2] Waiting for resource...
[Worker 2] Got resource!
[Worker 3] Waiting for resource...
[Worker 3] Got resource!
[Worker 4] Waiting for resource... (waits until one releases)
[Worker 1] Released resource!
[Worker 4] Got resource!
```

Example 3: Mutex – Mutual Exclusion

Scenario: Two tasks print to console. Mutex ensures clean output.

```
SemaphoreHandle_t printMutex;

void Printer(void *pv){
    char *name = (char*)pv;
    for(int i=0; i<5; i++){
        xSemaphoreTake(printMutex, portMAX_DELAY);
        printf("%s: Printing line %d\n", name, i+1);
        xSemaphoreGive(printMutex);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
    vTaskDelete(NULL);
}

void app_main(void){
    printMutex = xSemaphoreCreateMutex();
    xTaskCreate(Printer, "Task1", 2048, "Task1", 1, NULL);
    xTaskCreate(Printer, "Task2", 2048, "Task2", 1, NULL);
}
```

Output (Console)

```
Task1: Printing line 1
Task2: Printing line 1
Task1: Printing line 2
Task2: Printing line 2
Task1: Printing line 3
Task2: Printing line 3
...
```

Without mutex, lines would be mixed/garbled. Mutex ensures one task prints at a time.

Key Points

- Use **Binary Semaphore** for signaling between tasks or ISR and task.
- Use **Counting Semaphore** to manage multiple identical resources.
- Use **Mutex** for resource protection (shared data, peripherals, console).
- Always **GIVE** what you **TAKE**.
- Use **FromISR** APIs inside interrupts.

FreeRTOS BINARY SEMAPHORE – Simple Example

A Binary Semaphore has only two states: **AVAILABLE** (1) and **TAKEN** (0).

It is commonly used for **signaling** or synchronization between tasks or between an ISR and a task.

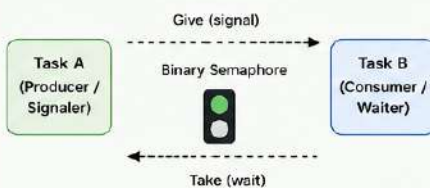
HOW IT WORKS

- When the semaphore is given (`xSemaphoreGive`), it becomes **AVAILABLE** (1).
- When a task takes it (`xSemaphoreTake`), it becomes **TAKEN** (0).
- If a task tries to take it while it's **TAKEN**, the task will block (wait) until it is given again.

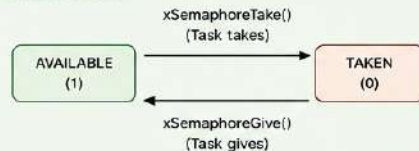
GENERAL EXAMPLE

Task B needs to do some work, but it must wait for a signal from Task A.

Task A does some work and then gives the signal.
Task B waits (blocks) on the semaphore.

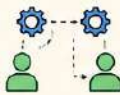


STATE FLOW



USE CASES (GENERAL)

- ✓ Task to task signaling
- ✓ Event notification (one-time event)
- ✓ ISR to task signaling
- ✓ Resource ready notification
- ✓ Button press / External event handling



SIMPLE CODE (General Example)

```
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

SemaphoreHandle_t binSem; // Handle for binary semaphore

// Task A: Gives the signal
void TaskA(void *pvParameters)
{
    while (1)
    {
        printf("[A] Doing some work...\n");
        vTaskDelay(pdMS_TO_TICKS(1000));

        printf("[A] Giving signal!\n");
        xSemaphoreGive(binSem); // Signal to Task B

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Task B: Waits for the signal
void TaskB(void *pvParameters)
{
    while (1)
    {
        printf("[B] Waiting for signal...\n");
        xSemaphoreTake(binSem, portMAX_DELAY); // Wait indefinitely

        printf("[B] Received signal! Doing work...\n");
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

int main(void)
{
    binSem = xSemaphoreCreateBinary(); // Create binary semaphore
    // Initially semaphore is TAKEN (0)

    xTaskCreate(TaskA, "TaskA", 2048, NULL, 1, NULL);
    xTaskCreate(TaskB, "TaskB", 2048, NULL, 1, NULL);

    vTaskStartScheduler();
    while (1) {}
}
```

Note: A newly created binary semaphore is in the **TAKEN (0)** state.
`xSemaphoreGive()` makes it **AVAILABLE (1)**.
`xSemaphoreTake()` makes it **TAKEN (0)**.

EXPECTED OUTPUT (Console)

```
[B] Waiting for signal...
[A] Doing some work...
[A] Giving signal!
[B] Received signal! Doing work...
[B] Waiting for signal...
[A] Doing some work...
[A] Giving signal!
[B] Received signal! Doing work...
[B] Waiting for signal...
[A] Doing some work...
[A] Giving signal!
[B] Received signal! Doing work...
...
```

EXPLANATION

- At start, Task B tries to take the semaphore but it is **TAKEN**, so Task B blocks (waits).
- Task A does some work, then gives the semaphore.
- Task B unblocks, receives the signal, does its work.
- Then Task B takes the semaphore again and blocks.
- The cycle repeats.

KEY POINTS

- ✓ Binary Semaphore = Signaling (not for resource counting).
- ✓ Use `xSemaphoreGive()` to send signal.
- ✓ Use `xSemaphoreTake()` to wait for signal.
- ✓ If the semaphore is not given, the waiting task will block.
- ✓ Very useful between tasks and between ISR and task.



FreeRTOS BINARY SEMAPHORE – Explained with Mangoes Example



A Binary Semaphore is like a **basket** that can hold only **1 mango**.

It is used for signaling – one task **gives** (puts a mango), another task **takes** (takes the mango).

STATES OF BASKET (SEMAPHORE)



Empty Basket
= **TAKEN (0)**
(No mango)



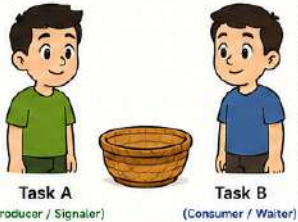
Basket with 1 Mango
= **AVAILABLE (1)**
(Has mango)

THE STORY

1. Initially

Basket is empty.

(Semaphore = **TAKEN (0)**)



Basket: Empty
TAKEN (0)

2. Task B tries to take

Task B wants a mango, but basket is empty. So Task B **waits**.



Basket: Empty
TAKEN (0)

3. Task A gives

Task A puts 1 mango in the basket (**gives signal**).



Basket: 1 Mango
AVAILABLE (1)

4. Task B takes

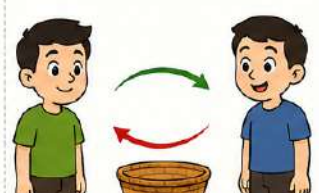
Task B sees the mango, **takes** it from the basket. He is unblocked and does his work.



Basket: Empty
TAKEN (0)

5. Cycle repeats

Now basket is empty again. Next time, Task B will wait until Task A gives a mango.



Basket: Empty
TAKEN (0)

KEY POINTS

- Basket can hold only **1 mango** → Binary (0 or 1).
- If basket is empty (**TAKEN**), Task B (waiter) must **wait**.
- When Task A (signaler) puts a mango (**gives**), basket becomes **AVAILABLE**.
- When Task B takes the mango (**takes**), basket becomes empty again (**TAKEN**).
- Used for **signaling / event** notification, not for counting multiple items.

HOW IT LOOKS IN CODE (Conceptual)

```
// Create binary semaphore (empty = TAKEN)
xSemaphoreHandle_t binSem = xSemaphoreCreateBinary();

// Task B (Waiter)
xSemaphoreTake(binSem, portMAX_DELAY); // waits if empty
// ... do work after getting the signal (mango)

// Task A (Signaler)
xSemaphoreGive(binSem); // puts mango (gives signal)
```

CONSOLE OUTPUT (Example)

```
[B] Waiting for mango...
[A] Putting mango in basket (give)!
[B] Got the mango! Doing work...
[B] Waiting for mango...
[A] Putting mango in basket (give)!
[B] Got the mango! Doing work...
[B] Waiting for mango...
...
(Cycle repeats)
```



SUMMARY:

Basket = Binary Semaphore

Mango = Signal (Event)

Put Mango = Give

Take Mango = Take

Two People = Two Tasks



FreeRTOS COUNTING SEMAPHORE – Explained with Mangoes Example



A **Counting Semaphore** is like a basket that can hold **multiple mangoes (n)**.

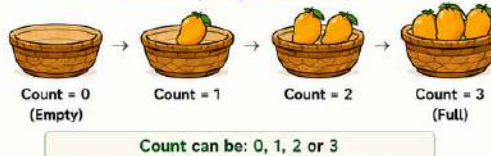
It is used to manage **multiple identical resources**.

In FreeRTOS, the count represents how many mangoes are currently in the basket.

- **TAKE** (`xSemaphoreTake`) → takes 1 mango (decrement).
- **GIVE** (`xSemaphoreGive`) → puts 1 mango (increment).
- If count is 0, the task blocks (waits).

OUR BASKET (Semaphore)

Basket Capacity (Max Count) = 3



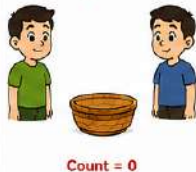
TWO PEOPLE = TWO TASKS



THE STORY (Step by Step)

1. Initially

Basket is empty.
Count = 0
(Semaphore = 0)



2. Task A gives

Task A puts 1 mango in the basket. (GIVE)



3. Task A gives again

Task A puts another mango. (GIVE)



4. Task B takes

Task B takes 1 mango from basket. (TAKE)



5. Task A gives

Task A puts 1 mango again. (GIVE)



6. Task B takes twice

Task B takes 1 mango. (TAKE again)



7. Task B tries to take

Basket is empty (count = 0). Task B wants a mango, but **must wait**. (Will block until Task A gives)



KEY POINTS

- Basket capacity (max count) = 3.
- Count shows how many mangoes are currently in the basket.
- **GIVE** → puts one mango (count++).
- **TAKE** → takes one mango (count--).
- If count is 0, **TAKE** will block (wait).
- Counting Semaphore is used when multiple identical resources are available.

SIMPLE CODE EXAMPLE (Conceptual)

```
// Create counting semaphore with max count = 3, initial count = 0
SemaphoreHandle_t countSem = xSemaphoreCreateCounting(3, 0);

// Task A - Producer (puts mango)
void TaskA(void *pvParameters)
{
    while (1)
    {
        vTaskDelay(pdMS_TO_TICKS(1000)); // gets mango from somewhere
        xSemaphoreGive(countSem); // put mango (give)
        printf("[A] Put mango. Count increased.\n");
    }
}

// Task B - Consumer (takes mango)
void TaskB(void *pvParameters)
{
    while (1)
    {
        printf("[B] Trying to take mango...\n");
        xSemaphoreTake(countSem, portMAX_DELAY); // wait if none available
        printf("[B] Took mango. Count decreased.\n");
        vTaskDelay(pdMS_TO_TICKS(1500)); // do something with mango
    }
}
```

CONSOLE OUTPUT (Example)

```
[A] Put mango. Count increased.
[A] Put mango. Count increased.
[B] Trying to take mango...
[B] Took mango. Count decreased.
[A] Put mango. Count increased.
[B] Trying to take mango...
[B] Took mango. Count decreased.
[B] Trying to take mango...
[B] Took mango. Count decreased.
[B] Trying to take mango...
(Blocked here, count = 0, waiting...)
[A] Put mango. Count increased.
[B] Took mango. Count decreased.
... (cycle continues)
```

When count becomes 0, Task B waits (blocks) until Task A gives (puts) a mango.

SUMMARY

Basket = Counting Semaphore
(Max Count = 3)

Mango = Resource / Signal
(1 unit)

Put Mango (GIVE)
→ count++

Take Mango (TAKE)
→ count--

Two People = Two Tasks
(Producer & Consumer)

If count = 0 →
Task waits (blocks)



FreeRTOS MUTEX (MUTUAL EXCLUSION) – Explained with Mangoes Example

A Mutex is a special type of Semaphore used for mutual exclusion. It ensures that only **ONE** task at a time can access a shared resource.

- Think of a basket of mangoes.
- Only one person can hold the basket key at a time.
- If the key is with someone else, others must wait.
- When the person finishes and returns the key, the next person can take it.

Mutex vs Binary Semaphore

Feature	Binary Semaphore	Mutex
Purpose	Signal / Event	Mutual Exclusion (Resource Protection)
Ownership	No owner	Has owner (which task locked it)
Priority Inheritance	No	Yes (prevents priority inversion)
Recursive Lock	No	Yes (same task can take multiple times)
Use Case	Task signaling	Protecting shared resources

THE STORY



KEY POINTS

- Mutex is for mutual exclusion (protect shared resources).
- Only one task can own (lock) the mutex at a time.
- If another task tries to take it, it will wait (block).
- Mutex has priority inheritance (helps avoid priority inversion).
- Always give (unlock) the mutex when done.
- Same task can take the mutex multiple times (recursive).



SIMPLE CODE EXAMPLE (Conceptual)

```
SemaphoreHandle_t xMutex; // Mutex handle

// Task A
void TaskA(void *pv){
    while(1){
        xSemaphoreTake(xMutex, portMAX_DELAY); // Lock
        printf("[A] Accessing mango basket...\n");
        vTaskDelay(pdMS_TO_TICKS(2000)); // Do work
        printf("[A] Done. Giving back the key.\n");
        xSemaphoreGive(xMutex); // Unlock
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Task B
void TaskB(void *pv){
    while(1){
        printf("[B] Waiting for basket...\n");
        xSemaphoreTake(xMutex, portMAX_DELAY); // Wait if locked
        printf("[B] Accessing mango basket...\n");
        vTaskDelay(pdMS_TO_TICKS(1500)); // Do work
        printf("[B] Done. Giving back the key.\n");
        xSemaphoreGive(xMutex); // Unlock
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// In main()
xMutex = xSemaphoreCreateMutex(); // Create mutex
xTaskCreate(TaskA, "TaskA", 2048, NULL, 2, NULL);
xTaskCreate(TaskB, "TaskB", 2048, NULL, 1, NULL);
```

CONSOLE OUTPUT (Example)

```
[B] Waiting for basket...
[A] Accessing mango basket...
[A] Done. Giving back the key.
[B] Accessing mango basket...
[B] Done. Giving back the key.
[A] Accessing mango basket...
[A] Done. Giving back the key.
[B] Accessing mango basket...
[B] Done. Giving back the key.
...
```

Notice how only one task accesses the basket (resource) at a time. The other waits until the key is returned.

WHY PRIORITY INHERITANCE?

- Scenario**
Low priority task holds the mutex.
- High priority task wants the mutex but must wait.
- Medium priority tasks keep running and may delay the low priority task.
- With Mutex: Low priority task temporarily inherits high priority and finishes faster → releases mutex → High priority task proceeds.

★ This avoids **Priority Inversion**.



SUMMARY



Basket
(Shared Resource)



= Key
(Mutex)



= Task A
(Producer/User)



= Task B
(Consumer/User)



Take Key (Lock) = xSemaphoreTake()

Give Key (Unlock) = xSemaphoreGive()

FreeRTOS QUEUES & MESSAGING – Inter-Task Communication (IPC)

WHAT IS A QUEUE?

- ▶ A Queue is like a line (FIFO) where messages (data) are sent by one task (**Sender**) and **received** by another task (**Receiver**).
- ▶ It is one of the most common mechanisms for Inter-Task Communication (IPC) in FreeRTOS.
- Think of it like passing messages in a box.
- Sender puts a message at the back.
- Receiver takes the message from the front.



MAIN QUEUE APIs

Sending (Enqueue)

- `xQueueSend()`
- `xQueueSendToBack()`
- `xQueueSendToFront()`
- `xQueueOverwrite()`

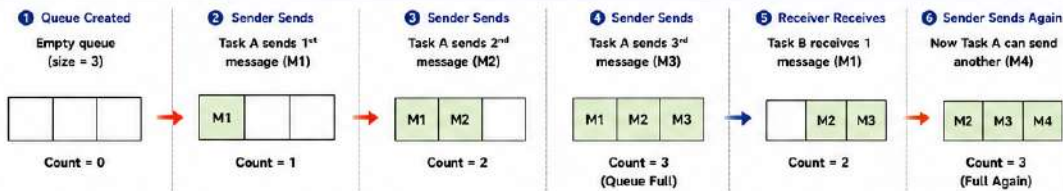
Receiving (Dequeue)

- `xQueueReceive()`
- `xQueuePeek()`

Useful Utility APIs

- `uxQueueMessagesWaiting()` – Number of messages in queue
- `uxQueueSpacesAvailable()` – Free spaces in queue
- `xQueueReset()` – Clear queue
- `vQueueDelete()` – Delete queue

HOW IT WORKS (EXAMPLE)



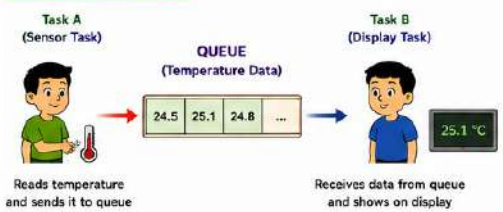
TYPES OF QUEUES

1. **Normal Queue**
Stores any type of data (structures, integers, etc.)
2. **Queue for Pointers**
Stores pointers (useful for large data).
3. **Queue Sets**
Allows waiting on multiple queues/semaphores.

KEY POINTS

- ✓ Queue is FIFO (First In, First Out).
- ✓ Sender does not need to know if receiver is ready.
- ✓ If queue is full, sender can wait (block).
- ✓ If queue is empty, receiver can wait (block).
- ✓ Ideal for data transfer between tasks.

EXAMPLE SCENARIO



SIMPLE CODE EXAMPLE

```
#include "FreeRTOS.h"
#include "queue.h"

// Create queue to hold 5 integers
QueueHandle_t xTempQueue;

void SenderTask(void *pvParameters)
{
    int temperature = 0;
    while (1)
    {
        temperature = readTemperature(); // Read sensor
        xQueueSend(xTempQueue, &temperature, portMAX_DELAY); // Send
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

void ReceiverTask(void *pvParameters)
{
    int receivedTemp;
    while (1)
    {
        if (xQueueReceive(xTempQueue, &receivedTemp, portMAX_DELAY) == pdPASS)
        {
            displayTemperature(receivedTemp); // Display on LCD/OLED
        }
    }
}
```

OUTPUT / FLOW (CONCEPTUAL)

```
[Sender] Read Temp: 24.5 -> Sent to Queue
[Sender] Read Temp: 25.1 -> Sent to Queue
[Sender] Read Temp: 24.8 -> Sent to Queue
[Receiver] Received Temp: 24.5 -> Displayed
[Receiver] Received Temp: 25.1 -> Displayed
[Receiver] Received Temp: 24.8 -> Displayed
[Sender] Read Temp: 25.3 -> Sent to Queue
[Receiver] Received Temp: 25.3 -> Displayed
...
```

Note:

If queue is full → Sender will wait (block) until space is available.
If queue is empty → Receiver will wait (block) until data arrives.

SUMMARY



- ✓ Queues provide reliable and safe data transfer between tasks.
- ✓ Helps in decoupling producer and consumer tasks.

FreeRTOS - MUTEX + COUNTING SEMAPHORE in One Example (Mango Basket Shop)

SCENARIO

There is a mango basket shop.

Basket Capacity = 3 mangoes (Counting Semaphore)

Only one person can access the basket at a time (Mutex)

People (Tasks) can take or put mangoes.

We will use:

- Counting Semaphore → to track how many mangoes are in the basket
- Mutex → to make sure only one person accesses the basket at a time



COUNTING SEMAPHORE

(Tracks Mangoes in Basket)

Initial count = 0 (Basket is empty)

Max count = 3 (Basket can hold 3 mangoes)

xSemaphoreGive(countSem)

→ Put mango (increase count)

xSemaphoreTake(countSem)

→ Take mango (decrease count)

Count shows how many mangoes are available

MUTEX (MUTUAL EXCLUSION)

(Protects the Basket)

xSemaphoreTake(mutexSem)

→ Lock the basket (exclusive access)

xSemaphoreGive(mutexSem)

→ Unlock the basket (release access)

Only one task can lock the basket at a time

TASKS (PEOPLE)

- Task A (Seller)
Puts mangoes in the basket
- Task B (Customer 1)
Takes mangoes from the basket
- Task C (Customer 2)
Takes mangoes from the basket
- Task D (Supplier)
Puts mangoes in the basket

STORY - HOW IT WORKS (Step by Step)

1. Start Basket is empty. Count = 0	2. Supplier (Task D) locks mutex and puts 1 mango.	3. D puts 2 more mangoes.	4. D unlocks mutex after putting.	5. Customer 1 (B) wants a mango. Locks mutex and takes 1 mango.	6. B unlocks mutex after taking.	7. Customer 2 (C) wants a mango. Locks mutex and takes 1 mango.	8. B tries to take another mango, but C holds the mutex, so B waits.	9. C unlocks mutex after taking.	10. B locks mutex now and takes the last mango.
Mutex: UNLOCKED (Free) Count: 0 / 3	Mutex: LOCKED (By D) Count: 1 / 3	Mutex: LOCKED (By D) Count: 3 / 3 (Full)	Mutex: UNLOCKED (Free) Count: 3 / 3	Mutex: LOCKED (By B) Count: 2 / 3	Mutex: UNLOCKED (Free) Count: 2 / 3	Mutex: LOCKED (By C) Count: 1 / 3	Mutex: LOCKED (By C) Count: 1 / 3	Mutex: UNLOCKED (Free) Count: 1 / 3	Mutex: LOCKED (By B) Count: 0 / 3 (Empty)

CODE (Conceptual)

```
SemaphoreHandle_t mutexSem; // Mutex for basket access
SemaphoreHandle_t countSem; // Counting semaphore for mango count (0..3)

void initSystem()
{
    mutexSem = xSemaphoreCreateMutex(); // Create mutex
    countSem = xSemaphoreCreateCounting(3, 0); // Max 3 mangoes, initial 0
}

// Put mango in basket (Supplier / Seller)
void putMango()
{
    xSemaphoreTake(mutexSem, portMAX_DELAY); // Lock basket
    // --- Critical Section: Access Basket ---
    xSemaphoreGive(countSem); // Increase count (put mango)
    printf("Mango put.\n");
    xSemaphoreGive(mutexSem); // Unlock basket
}

// Take mango from basket (Customer)
void takeMango(const char *name)
{
    // 1. Wait until at least 1 mango is available
    xSemaphoreTake(countSem, portMAX_DELAY); // Decrease count (take mango)

    // 2. Lock basket to access safely
    xSemaphoreTake(mutexSem, portMAX_DELAY); // Lock basket

    // --- Critical Section: Access Basket ---
    printf("%s took a mango.\n", name);
    xSemaphoreGive(mutexSem); // Unlock basket
}
```

SAMPLE OUTPUT (Possible)

```
Start: Basket empty. Count = 0
Supplier(D) put mango.
Supplier(D) put mango.
Supplier(D) put mango.
Count = 3 (Basket Full)
Customer1(B) took a mango.
Count = 2
Customer2(C) took a mango.
Count = 1
Customer1(B) took a mango.
Count = 0 (Basket Empty)
```

KEY TAKEAWAYS

- ✓ Counting Semaphore
→ Tracks how many mangoes are in the basket.
→ Allows up to N tasks to put/take (based on count).
- ✓ Mutex
→ Ensures only one task accesses the basket at a time (mutual exclusion).
→ Prevents data corruption and conflicts.
- ✓ Together
→ Semaphore manages "quantity" (how many)
→ Mutex manages "access" (who and when)

★ This combination is powerful for real-world systems (buffers, resources, devices, queues, etc.)

SUMMARY



FREE RTOS – DEADLOCK EXPLAINED

(Mango & Keys Example) 🥭

WHAT IS DEADLOCK?

Deadlock is a situation where two or more tasks are waiting for each other forever, so none of them can proceed.



System gets stuck. No task can continue.

OUR RESOURCES (MUTEXES)

Basket Key
(Mutex 1)



Used to access
the Basket



Mango Box Key
(Mutex 2)



Used to access
the Mango Box



OUR TASKS (PEOPLE)

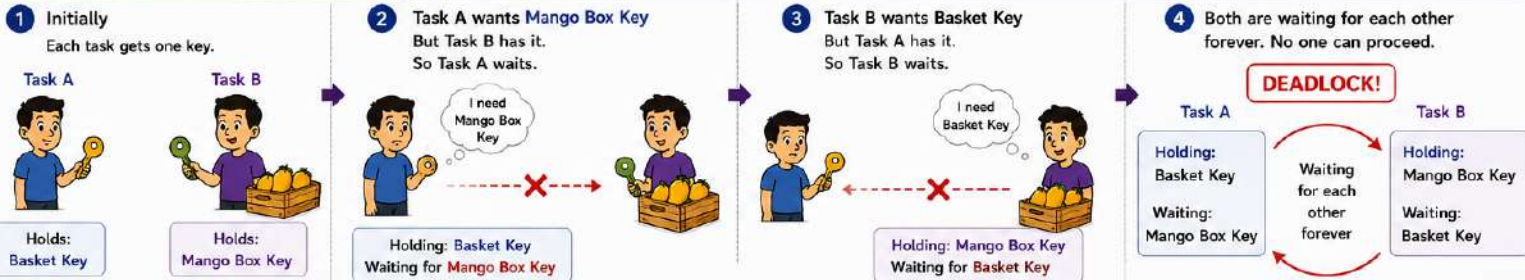
Task A
(Person A)



Task B
(Person B)



HOW DEADLOCK HAPPENS (STEP BY STEP)



CODE EXAMPLE (FreeRTOS)

Task A Code

```
xSemaphoreTake(basketMutex, portMAX_DELAY);  
// Got Basket Key  
// Do some work...  
xSemaphoreTake(boxMutex, portMAX_DELAY);  
// Waiting here forever for Mango Box Key  
  
// This line is never reached  
xSemaphoreGive(boxMutex);  
xSemaphoreGive(basketMutex);
```

Task B Code

```
xSemaphoreTake(boxMutex, portMAX_DELAY);  
// Got Mango Box Key  
// Do some work...  
xSemaphoreTake(basketMutex, portMAX_DELAY);  
// Waiting here forever for Basket Key  
  
// This line is never reached  
xSemaphoreGive(basketMutex);  
xSemaphoreGive(boxMutex);
```

POSSIBLE CONSOLE OUTPUT

```
Task A : Took Basket Key  
Task B : Took Mango Box Key  
  
Task A : Waiting for Mango Box Key...  
Task B : Waiting for Basket Key...  
..... (nothing more happens)  
..... (both are blocked)  
  
SYSTEM STUCK (DEADLOCK)
```

HOW TO AVOID DEADLOCK

- ✓ Always acquire multiple resources in the same order.

Example Order (Always follow this):

- 1 Take Basket Key (Mutex 1)
- 2 Take Mango Box Key (Mutex 2)
- 3 Do your work
- 4 Release Mango Box Key (Mutex 2)
- 5 Release Basket Key (Mutex 1)



Consistent order prevents deadlock!

SUMMARY



Mutex
(Basket Key)



Mutex
(Mango Box Key)



If taken in different order
by tasks



Deadlock
Happens



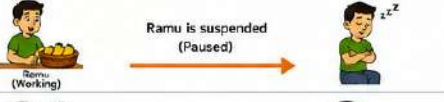
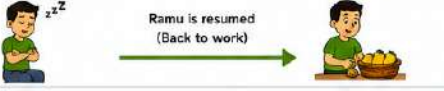
Tasks wait forever
System stuck 😞

REMEMBER

Deadlock = Each task holds a resource and waits for another held by others.



FreeRTOS – TASK CONTROL FUNCTIONS (Simple Mango Examples) 🥭

FUNCTION	WHAT IT DOES	MANGO EXAMPLE (Simple)	VISUAL STORY	CODE EXAMPLE (Conceptual)
xTaskCreate() 	Creates a new task and adds it to the system.	Ramu (a new worker) joins the mango shop to help.		<pre>xTaskCreate(MangoTask, // Task function "Ramu", // Name 1024, // Stack size NULL, // Parameters 2, // Priority NULL // Task handle);</pre>
vTaskDelete() 	Deletes the calling task. It removes itself from the system.	Ramu finishes work and leaves the shop.		<pre>vTaskDelete(NULL); // Task deletes itself</pre>
vTaskDelay() 	Puts the task into Blocked state for the specified time.	Ramu takes a short break for some time.		<pre>vTaskDelay(2000 / portTICK_PERIOD_MS); // Delay for 2 seconds</pre>
vTaskDelayUntil() 	Puts the task into Blocked state until the next absolute time. Good for periodic work.	Ramu packs mangoes every 5 seconds like clockwork.		<pre>TickType_t lastWakeTime = xTaskGetTickCount(); const TickType_t period = 5000 / portTICK_PERIOD_MS; for(;;) { DoPacking(); vTaskDelayUntil(&lastWakeTime, period); // every 5 sec }</pre>
vTaskSuspend() 	Suspends (pauses) a task. It will not run until resumed.	Ramu is asked to pause work (goes on hold).		<pre>vTaskSuspend(xTaskHandle); // Suspend the task</pre>
vTaskResume() 	Resumes a suspended task so it can run again.	Ramu is called back to continue work.		<pre>vTaskResume(xTaskHandle); // Resume the suspended task</pre>
vTaskPrioritySet() 	Changes the priority of a task.	Ramu gets more important work, so his priority is increased.		<pre>vTaskPrioritySet(xTaskHandle, 3); // Set new priority (e.g., 3)</pre>



In Short:

Create a task (xTaskCreate) → Do work → Delay if needed (vTaskDelay / vTaskDelayUntil) → Pause (vTaskSuspend) → Resume (vTaskResume) → Change priority when required (vTaskPrioritySet) → Delete when done (vTaskDelete).



All examples use a simple Mango Shop story for easy understanding!

xTaskCreate() – PARAMETERS EXPLAINED (Mango Shop Example) 🍌

💡 **xTaskCreate()** is used to **create a new task** (a new worker) and **add it to the FreeRTOS system**.

Syntax: `xTaskCreate(TaskFunction, Name, StackSize, Parameters, Priority, TaskHandle);`

Parameter	What it means	Mango Shop Example	Example Value
1 TaskFunction	Function that contains the task code.	 The job/role of the worker.	MangoTask
2 Name	Name of the task (unique as ID).	 Worker's name.	"RamuTask"
3 StackSize	Amount of stack memory (in words) for the task.	 How big a basket (worker needs to carry things).	1024
4 Parameters	Input given to the task when it starts.	 Tools/Things given to the worker to do the job.	(void*) &data
5 Priority	Priority of the task. Higher value = higher priority.	 How important the worker is in the shop.	2 (medium)
6 TaskHandle	Returns a handle (identification) of the created task.	 ID card of the worker (returned by RTOS).	&xHandle

EXAMPLE

```
TaskHandle_t xHandle;
```

```
xTaskCreate(  
1 MangoTask,           // TaskFunction  
2 "RamuTask",          // Name  
3 1024,                // StackSize  
4 (void*)&data,          // Parameters  
5 2,                   // Priority  
6 &xHandle              // TaskHandle  
);
```

What happens?

- ✓ A new task (Ramu) is created.
- ✓ Ramu gets a basket (stack) of 1024 size.
- ✓ He gets tools (data) to do his job.
- ✓ He works with priority 2.
- ✓ RTOS gives an ID (handle) of Ramu.

MANGO SHOP STORY



★ **Remember:** Choose the Job (TaskFunction) → Give a Name (Name) → Give Basket Size (StackSize) → Give Tools (Parameters) → Set Priority (Priority) → Get ID Card (TaskHandle)

💡 Using **xTaskCreate()**, we bring a new worker (task) into the system and let him start working! 🍌

xTaskCreate() – Parameters Explained (Mango Shop Example)

```
xTaskCreate( TaskFunction_t pxTaskCode, const char * const pcName,
            uint16_t usStackDepth, void *pvParameters,
            UBaseType_t uxPriority, TaskHandle_t *pxCreatedTask );
```



Example Call

```
xTaskCreate( MangoTask,           // 1 Function
            "SellerTask",         // 2 Name
            1024,                 // 3 Stack size
            &mangoCount,          // 4 Parameters
            3,                   // 5 Priority
            &SellerTaskHandle     // 6 Task handle
        );
```

In Short

xTaskCreate tells FreeRTOS
 → What work to do
 → Worker name
 → How much memory to use
 → What data to give
 → How important the worker is
 → And gives back a handle to control that worker



Parameter	What it means	Mango Shop Example	Example Value
1 pxTaskCode	Function (code) that the task will run.	 The work the worker has to do (e.g., sell mangoes)	MangoTask
2 pcName	Name of the task (Just for identification).	 Worker's name in the shop	"SellerTask"
3 usStackDepth	Stack size (memory) needed by the task.	 How much space (worker needs to keep his stuff)	1024 (words)
4 pvParameters	Data to pass to the task when it starts.	 Initial information (worker gets when joining)	(void *) &mangoCount
5 uxPriority	Priority of the task (higher value = higher priority).	 Important worker gets higher priority to serve first	3 (High)
6 pxCreatedTask	Returns a handle to the created task (NULL if failed).	 Reference/handle to the worker	&SellerTaskHandle

★ xTaskCreate() = Hire a new worker with name, job, space, data, priority and get his ID (handle)